

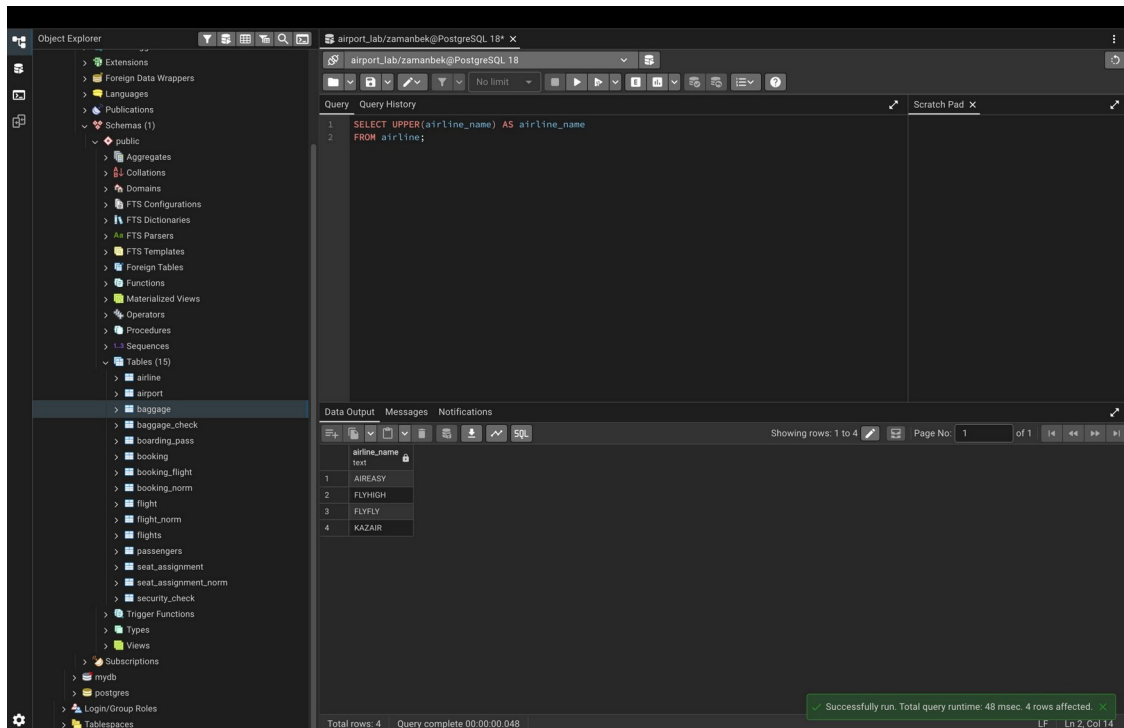
Lab Report #4

Task 1. Retrieve all airline names in uppercase.

SQL Query:

```
SELECT UPPER(airline_name) AS airline_name
FROM airline;
```

Query Result:



The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' pane showing a database schema with various tables and functions. The 'airline' table is selected. The main window displays the SQL query: `SELECT UPPER(airline_name) AS airline_name FROM airline;`. Below the query editor is the 'Data Output' pane, which shows the results of the query in a table format. The table has one column, 'airline_name', and four rows of data: 'AIREASY', 'FLYHIGH', 'FLYFLY', and 'KAZAIR'. The status bar at the bottom indicates 'Total rows: 4' and 'Query complete 00:00:00.048'.

airline_name
AIREASY
FLYHIGH
FLYFLY
KAZAIR

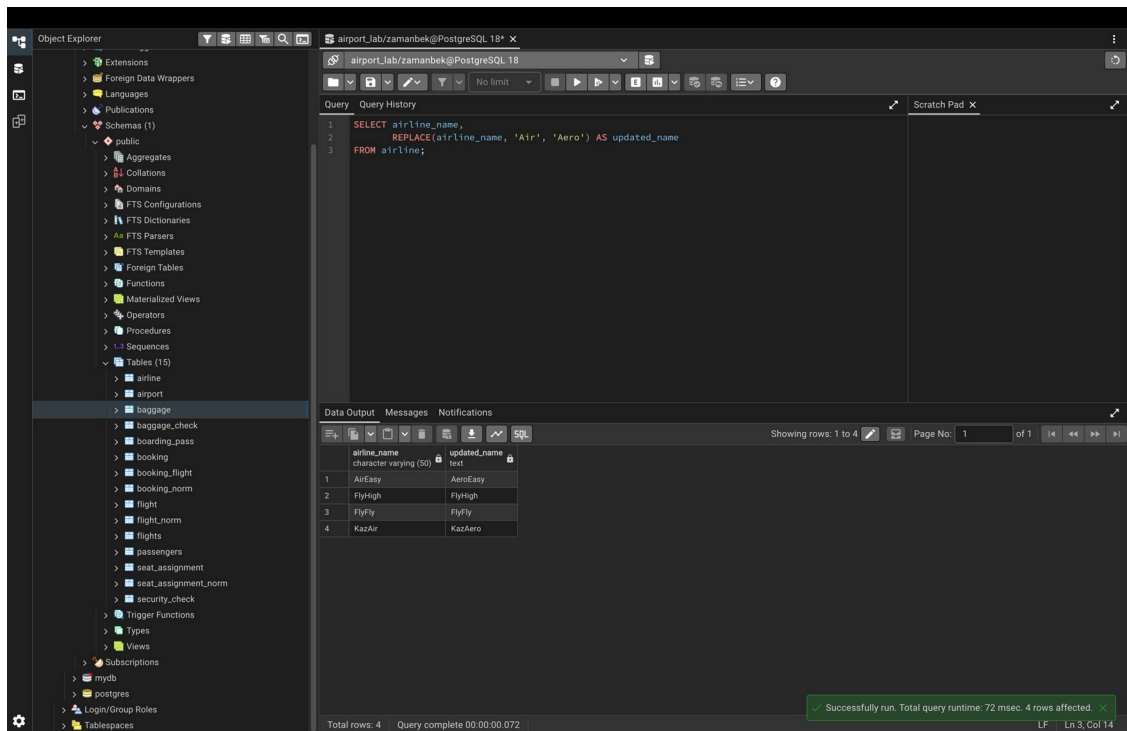
Task 2. Replace any occurrence of the word "Air" in airline names with "Aero".

SQL Query:

```
SELECT airline_name,
       REPLACE(airline_name, 'Air', 'Aero') AS updated_name
FROM airline;
```

REPLACE() is used inside a SELECT, so the table itself is not changed; it only shows what the renamed values would look like.

Query Result:



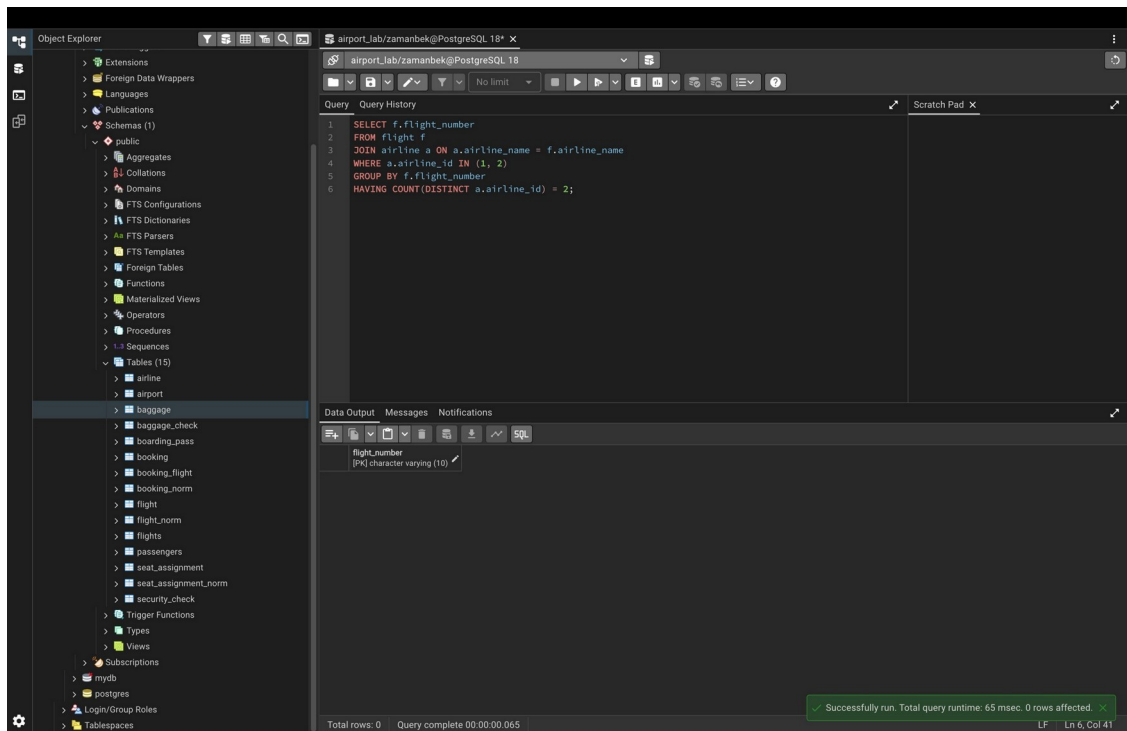
Task 3. Find all flight numbers that correspond to both airline 1 and airline 2.

SQL Query:

```
SELECT f.flight_number  
FROM flight f  
JOIN airline a ON a.airline_name = f.airline_name  
WHERE a.airline_id IN (1, 2)  
GROUP BY f.flight_number  
HAVING COUNT(DISTINCT a.airline_id) = 2;
```

The "flight" table only stores airline_name (no airline_id), so it is joined to "airline" to filter by id. HAVING COUNT(DISTINCT airline_id) = 2 keeps only flight numbers that appear under both airline 1 and airline 2.

Query Result:



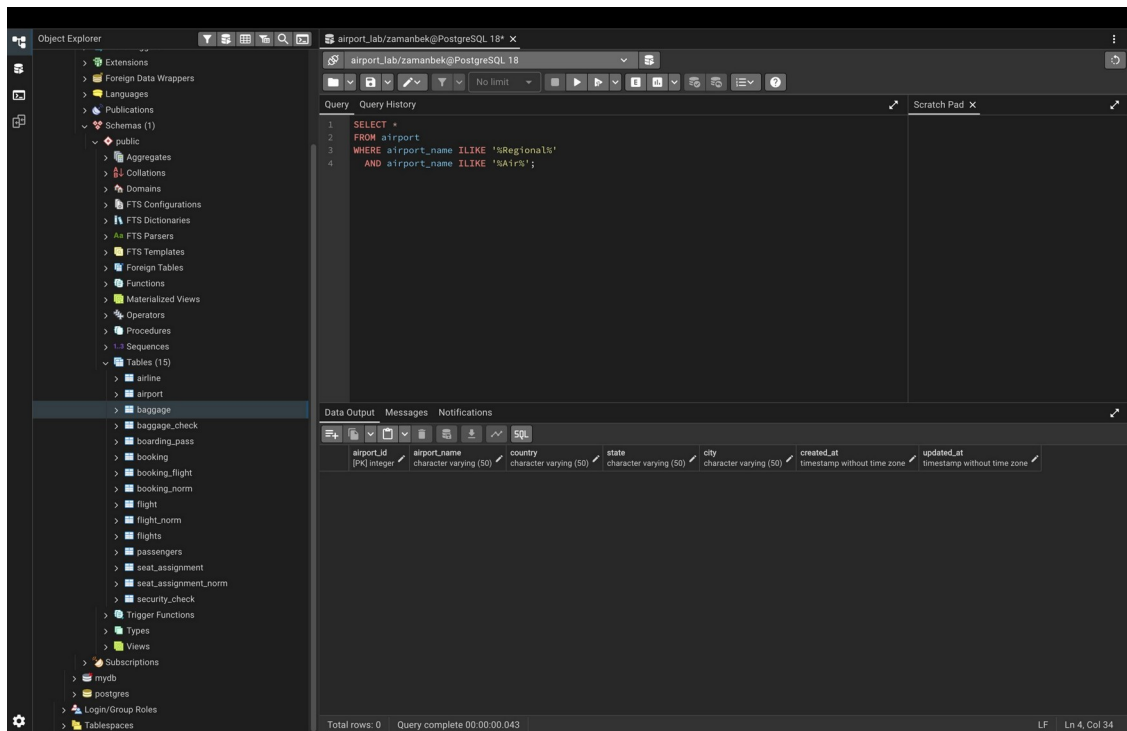
Task 4. Retrieve airports that contain the word "Regional" and "Air" in their names.

SQL Query:

```
SELECT *
FROM airport
WHERE airport_name ILIKE '%Regional%'
AND airport_name ILIKE '%Air%';
```

ILIKE is case-insensitive LIKE. The task text has "Reginal" — if your airport_name values actually use that spelling, change "%Regional%" to "%Reginal%".

Query Result:



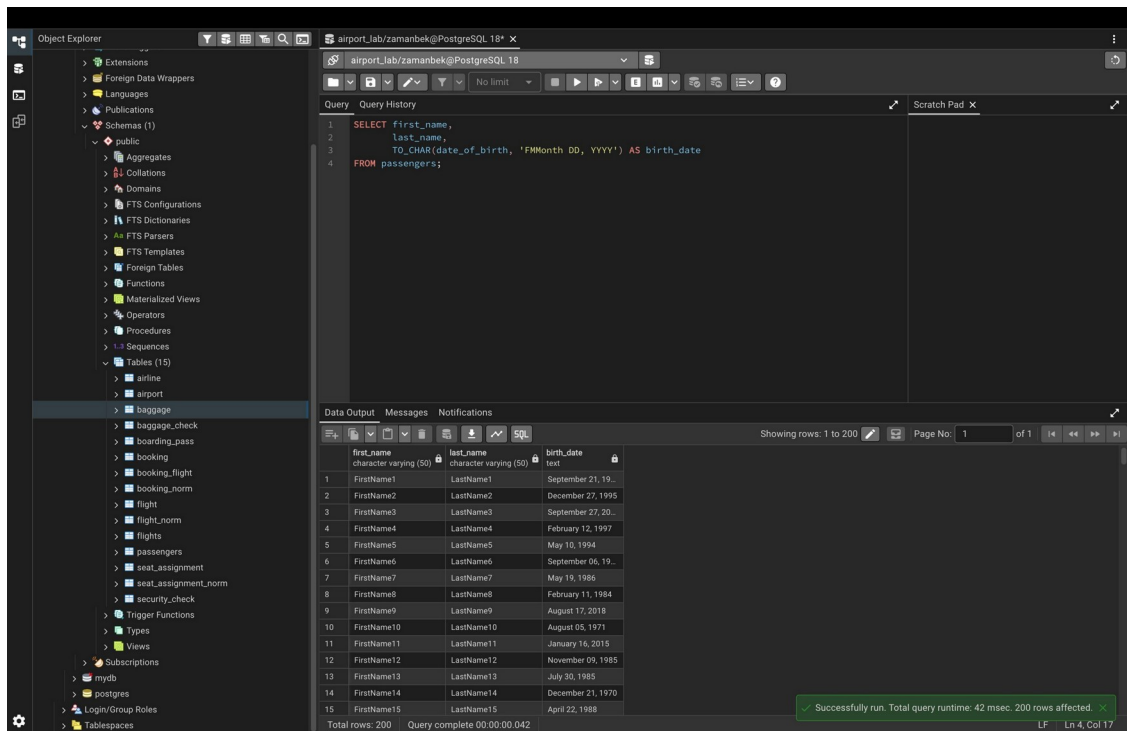
Task 5. Retrieve passenger names and format their birth dates as 'Month DD, YYYY'.

SQL Query:

```
SELECT first_name,
       last_name,
       TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date
FROM passengers;
```

TO_CHAR() formats a date as text. The FM prefix removes the extra padding spaces that Month/DD would otherwise add.

Query Result:



Task 6. Find flight numbers that have been delayed based on the actual arrival time.

SQL Query:

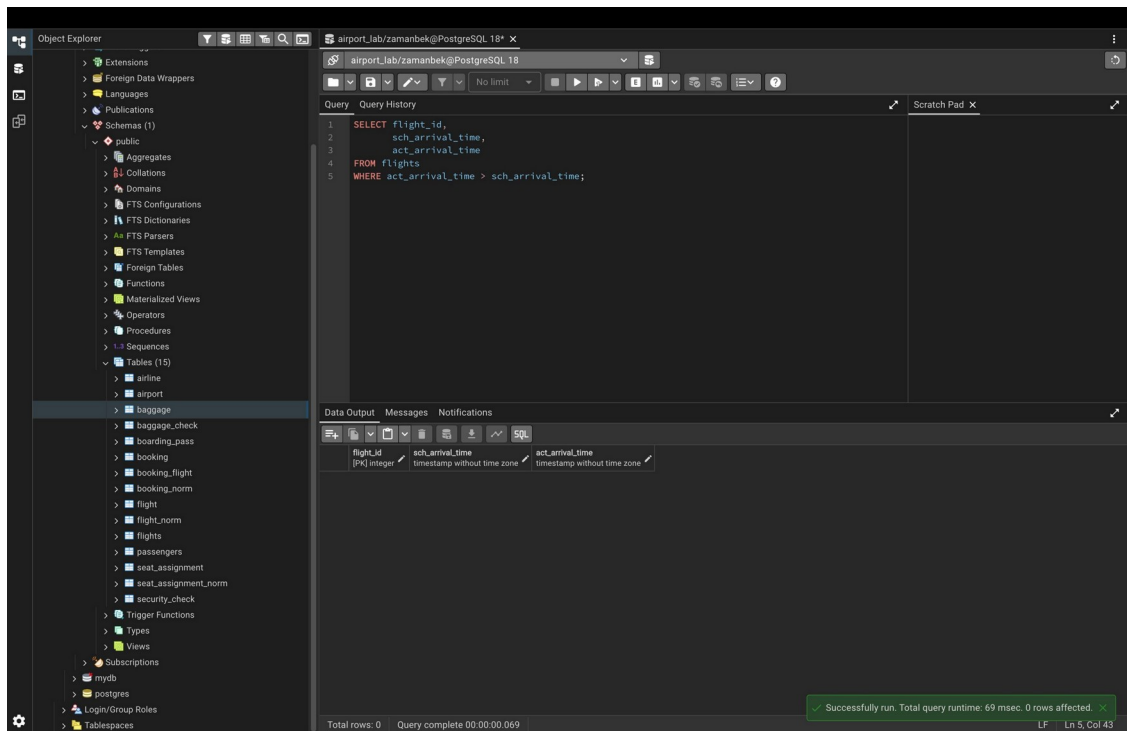
```

SELECT flight_id,
       sch_arrival_time,
       act_arrival_time
FROM flights
WHERE act_arrival_time > sch_arrival_time;

```

The "flights" table has no flight_number column, only flight_id, so flight_id is used as the flight identifier here.

Query Result:



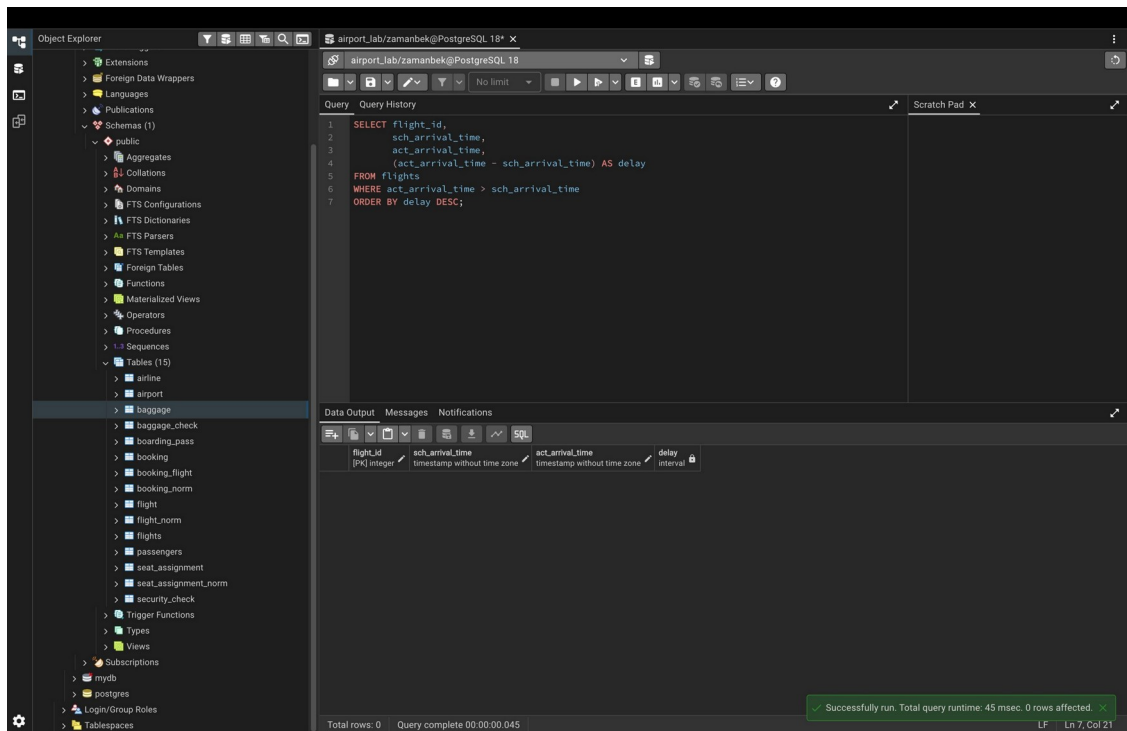
Task 7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

SQL Query:

```
SELECT flight_id,  
       sch_arrival_time,  
       act_arrival_time,  
       (act_arrival_time - sch_arrival_time) AS delay  
FROM flights  
WHERE act_arrival_time > sch_arrival_time  
ORDER BY delay DESC;
```

Same condition as Task 6, but this version also computes and shows how long each flight was delayed by subtracting the two timestamps.

Query Result:



Task 8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.

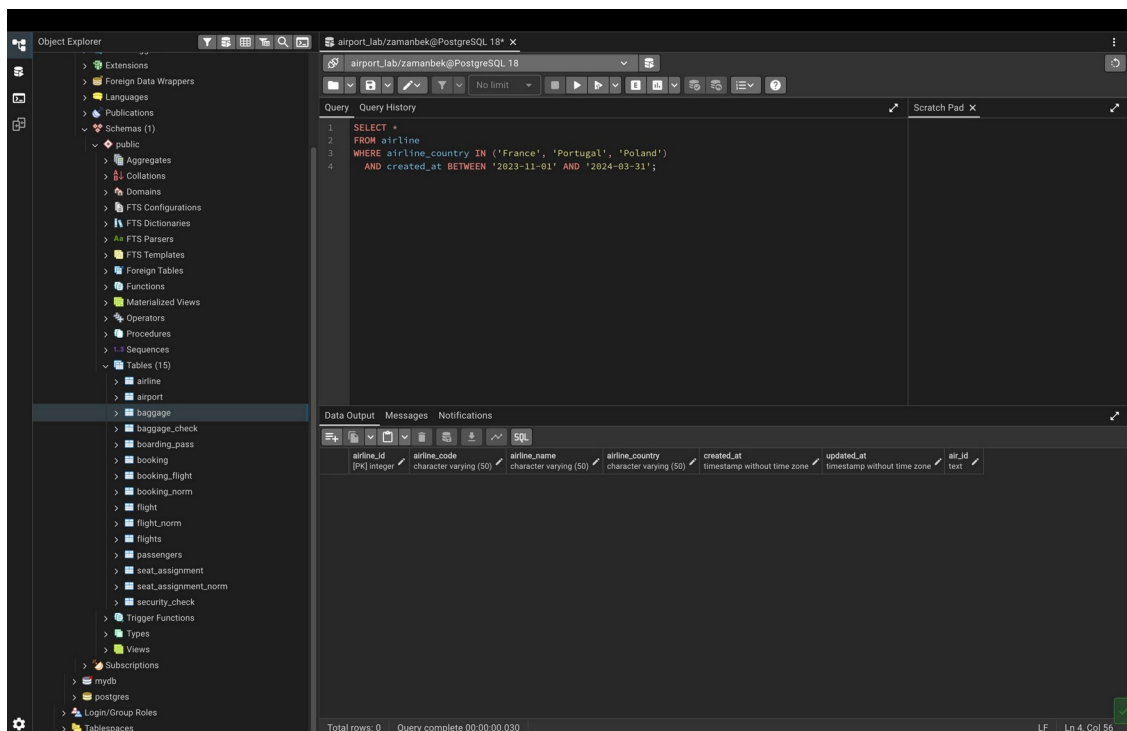
SQL Query:

```

SELECT *
FROM airline
WHERE airline_country IN ('France', 'Portugal', 'Poland')
AND created_at BETWEEN '2023-11-01' AND '2024-03-31';

```

Query Result:

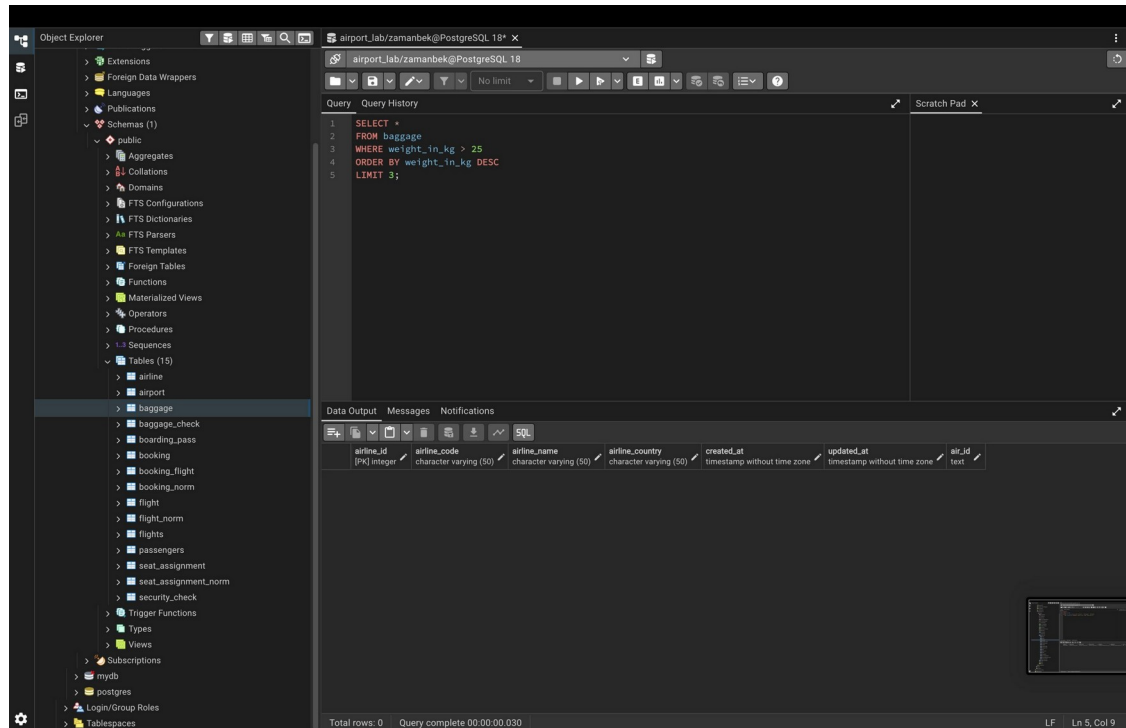


Task 9. Find top3 overweighted baggage with more than 25kg.

SQL Query:

```
SELECT *
FROM baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC
LIMIT 3;
```

Query Result:



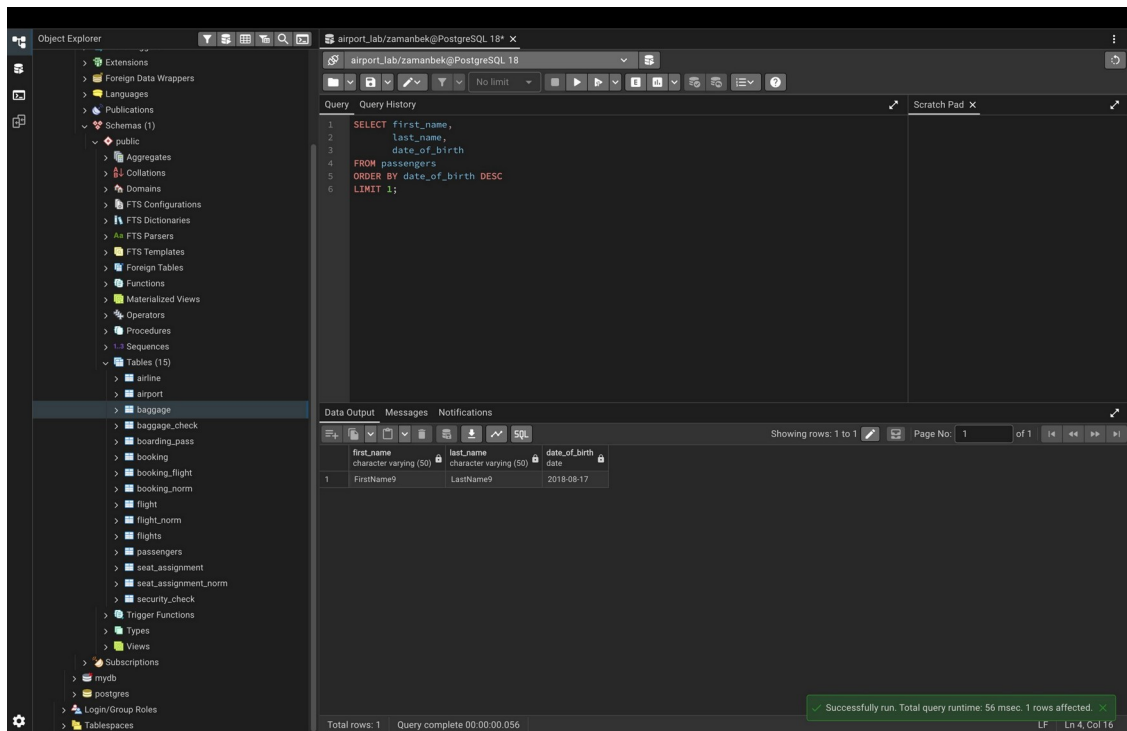
Task 10. Find the youngest passengers' full name.

SQL Query:

```
SELECT first_name,
       last_name,
       date_of_birth
FROM passengers
ORDER BY date_of_birth DESC
LIMIT 1;
```

The youngest passenger has the most recent date_of_birth. Remove LIMIT 1 (or use LIMIT with a higher number) if you need to see ties.

Query Result:



Task 11. Find the cheapest booking price on each booking platform.

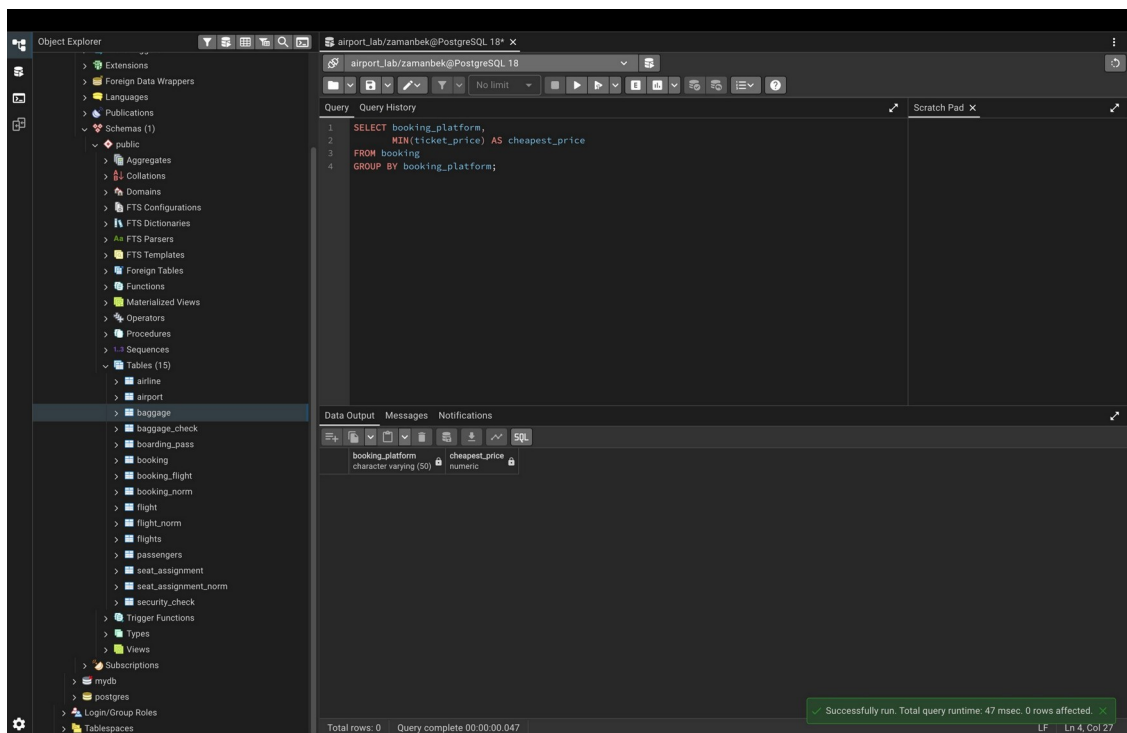
SQL Query:

```

SELECT booking_platform,
       MIN(ticket_price) AS cheapest_price
FROM booking
GROUP BY booking_platform;

```

Query Result:



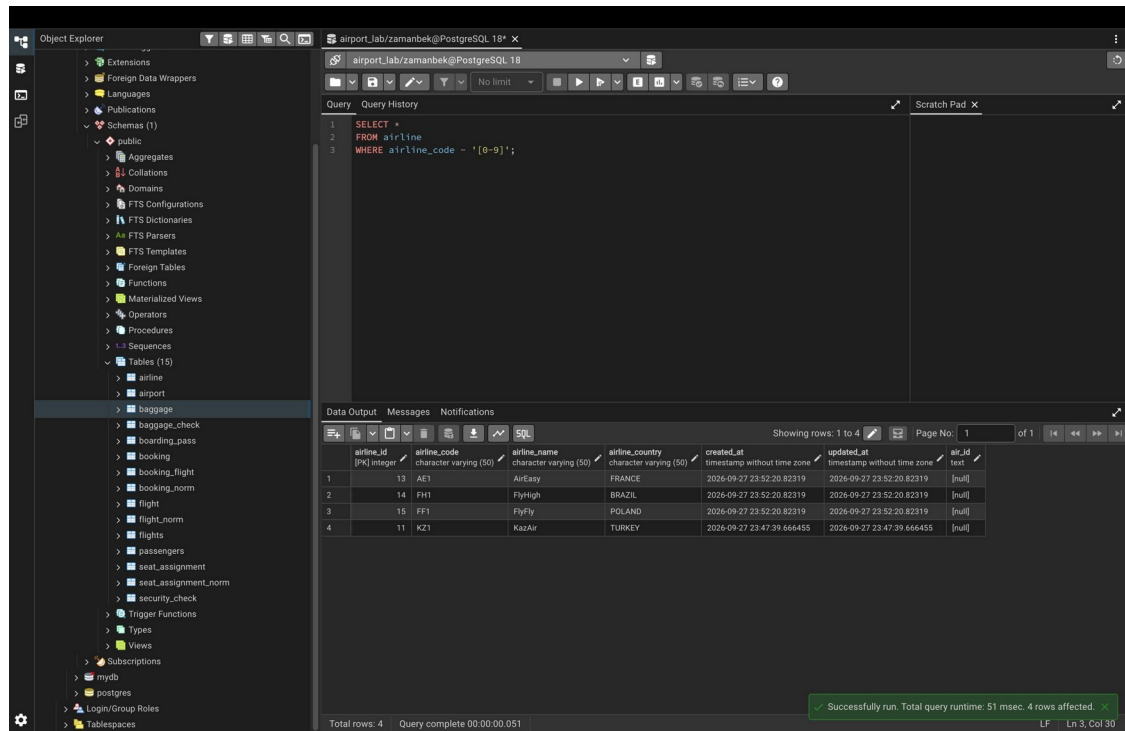
Task 12. Return airlines whose airline_code contains a digit.

SQL Query:

```
SELECT *  
FROM airline  
WHERE airline_code ~ '[0-9]';
```

The ~ operator matches a regular expression in PostgreSQL. '[0-9]' matches any single digit anywhere in the string.

Query Result:



The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' pane showing a database schema with various tables and views. The main window displays the 'Query History' tab with the following SQL query:

```
1 SELECT *  
2 FROM airline  
3 WHERE airline_code ~ '[0-9]';
```

Below the query, the 'Data Output' tab shows the results of the query. The results are displayed in a table with the following columns: airline_id, airline_code, airline_name, airline_country, created_at, updated_at, and air_id. The table contains 4 rows of data.

airline_id	airline_code	airline_name	airline_country	created_at	updated_at	air_id
13	AE1	AirEasy	FRANCE	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
14	FH1	FlyHigh	BRAZIL	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
15	FF1	FlyFly	POLAND	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
11	KZ1	KazAir	TURKEY	2026-09-27 23:47:39.666455	2026-09-27 23:47:39.666455	[null]

At the bottom of the IDE, a status bar indicates 'Total rows: 4' and 'Query complete 00:00:00.051'. A green message box at the bottom right states 'Successfully run. Total query runtime: 51 msec. 4 rows affected.'

Task 13. List the top5 most recently created airlines.

SQL Query:

```
SELECT *  
FROM airline  
ORDER BY created_at DESC  
LIMIT 5;
```

Query Result:

The screenshot shows the PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with the 'airline' table selected under the 'public' schema. The main query editor contains the following SQL query:

```

1 SELECT *
2 FROM airline
3 ORDER BY created_at DESC
4 LIMIT 5;

```

The 'Data Output' pane at the bottom shows the results of the query, displaying 4 rows. The status bar at the bottom indicates 'Successfully run. Total query runtime: 55 msec. 4 rows affected.'

airline_id [PK] integer	airline_code character varying (50)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	air_id text
13	AE1	AirEasy	FRANCE	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
14	FH1	FlyHigh	BRAZIL	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
15	FF1	FlyFly	POLAND	2026-09-27 23:52:20.82319	2026-09-27 23:52:20.82319	[null]
11	KZ1	KazAir	TURKEY	2026-09-27 23:47:39.666455	2026-09-27 23:47:39.666455	[null]

Task 14. Return all rows where booking_id is between 200 and 300 inclusive and check_result <> 'Checked'.

SQL Query:

```

SELECT *
FROM baggage_check
WHERE booking_id BETWEEN 200 AND 300
AND check_result <> 'Checked';

```

Query Result:

The screenshot shows the PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with the 'baggage_check' table selected under the 'public' schema. The main query editor contains the following SQL query:

```

1 SELECT *
2 FROM baggage_check
3 WHERE booking_id BETWEEN 200 AND 300
4 AND check_result <> 'Checked';

```

The 'Data Output' pane at the bottom shows the results of the query, displaying 0 rows. The status bar at the bottom indicates 'Successfully run. Total query runtime: 60 msec. 0 rows affected.'

baggage_check_id [PK] integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
-------------------------------	-------------------------------------	--	--	--------------------	----------------------

Task 15. Baggage checks where updated_at is in the same month as created_at but occurs earlier than created_at.

SQL Query:

```
SELECT *  
FROM baggage_check  
WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)  
AND updated_at < created_at;
```

Query Result:

The screenshot displays a PostgreSQL IDE interface. On the left, the 'Object Explorer' pane shows a database schema with various tables and views. The 'baggage_check' table is selected. The main pane shows the SQL query being executed. The 'Data Output' pane at the bottom shows the results of the query, which are empty. A status bar at the bottom indicates 'Total rows: 0' and 'Query complete 00:00:00.049'. A green message box at the bottom right states 'Successfully run. Total query runtime: 49 msec. 0 rows affected.'

Object Explorer

airpot_lab/zamanbek@PostgreSQL 18

Query

```
1 SELECT *  
2 FROM baggage_check  
3 WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at)  
4 AND updated_at < created_at;
```

Scratch Pad

Data Output

baggage_check_id	check_result	created_at	updated_at	booking_id	passenger_id
------------------	--------------	------------	------------	------------	--------------

Total rows: 0 Query complete 00:00:00.049

Successfully run. Total query runtime: 49 msec. 0 rows affected.